Network Intrusion Detection System Based on Singular Value Decomposition (SVD)

Project Team:
Andrei Oroian Patrick Rada Teodor Tita

Scientific Advisor:
Lect. Dr. Mihai Bucătaru

Faculty of Mathematics and Computer Science, University of Bucharest
Field of Computer Science, Specialization in Computers and Information Technology

May 31, 2026

Contents

1 Mathematical Description of SVD and Its Application to Anomaly Detection **2**
1.1 Using SVD for Intrusion Detection 2

2 Implementation of the SVD Algorithm **2**
2.1 Function Implementation and Numerical Stability 3
2.2 Numerical Validation 3

3 Comparison with NumPy Routines **3**

4 Dataset and Experimental Results **4**
4.1 Data Preprocessing 4
4.2 Evaluation Metrics 4

5 Result Interpretation: Visual Analysis and Practical Implications **4**
5.1 Graphical Analysis 4
5.2 Strengths and Limitations 4

6 Conclusion **5**

1 Mathematical Description of SVD and Its Application to Anomaly Detection

Singular Value Decomposition (SVD) allows us to factor any matrix $A \in \mathbb{R}^{m \times n}$ into the form:

$$A = U \Sigma V^T \tag{1}$$

To better understand the dimensions and structure, the decomposition can be visualized as follows:

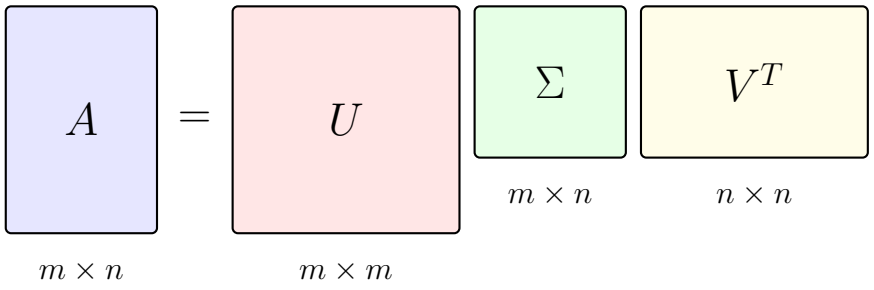


Figure 1: Geometric representation of matrix dimensions in SVD.

Here, U and V are orthogonal matrices containing the singular vectors, and Σ is the diagonal matrix of singular values sorted in descending order.

1.1 Using SVD for Intrusion Detection

The core idea of the project is to employ SVD to model normal network traffic behavior. The procedure consists of the following steps:

1. **Training:** We compute the SVD exclusively on normal traffic data. The first k columns of V define the principal subspace of clean network traffic.

2. **Projection:** When evaluating a new connection (x), we project it onto this subspace and reconstruct it using $\hat{x} = V_k V_k^T x$.

3. **Decision:** We evaluate the Euclidean distance between the original and reconstructed vectors ($\|x - \hat{x}\|_2$). Cyberattacks deviate from normal network structure and cannot be reconstructed accurately. Consequently, they yield a high reconstruction error, allowing detection via a calibrated threshold.

2 Implementation of the SVD Algorithm

To adhere to the requirement of not using built-in library routines (such as NumPy functions for diagonalization or vector norm computation), the algorithm was implemented from scratch.

We adopted the classical approach using the sample covariance matrix $M = A^T A$ and iteratively applied the Power Iteration method with Hotelling's deflation. This is well-suited for truncated SVD, where only the top k principal components are required rather than the full decomposition.

2.1 Function Implementation and Numerical Stability

The implementation incorporates several numerical strategies to maintain stability:

1. **Custom Norm Function:** Rather than calling `np.linalg.norm`, we implemented a dedicated routine `norm(v)` that accumulates squared components and extracts the square root.

2. **Matrix Scaling to Prevent Overflow:** Power iteration repeatedly multiplies a matrix by a vector. On real-world datasets, intermediate values can escalate rapidly, causing floating-point overflow. To mitigate this, the entire matrix is initially normalized by its maximum absolute entry (scaling factor). Iterations proceed on the scaled matrix, and the computed eigenvalue is subsequently rescaled.

3. **Resolving Sign Oscillations:** A common issue in power iteration is alternating sign flips between v and $-v$ at consecutive iterations, leading to non-termination. We mitigated this by checking convergence against two termination conditions: small norm differences ($\|v^{(t)} - v^{(t-1)}\| < \varepsilon$) as well as small norm sums ($\|v^{(t)} + v^{(t-1)}\| < \varepsilon$).

4. **Matrix Deflation:** Once an eigenvalue-eigenvector pair is found, its outer product contribution is subtracted from the working matrix. The subsequent iteration will then converge to the next dominant component.

2.2 Numerical Validation

We included a test harness within the `main` entry point of the SVD script to verify whether the reconstruction $U\Sigma V^T$ matches the input matrix A. The reconstructed matrix matched the original data within a numerical tolerance of 10^{-8}, confirming the correctness of the custom decomposition routine.

3 Comparison with NumPy Routines

We benchmarked the implementation against `numpy.linalg.svd`, which relies on precompiled C and Fortran implementations (LAPACK).

Table 1: Comparative Technical Analysis

Metric	Our Implementation	NumPy (LAPACK)
Built-in routines used	0	Full (black-box)
Reconstruction accuracy	High (10^{-8})	Machine precision
Overflow protection	Explicit (scaling)	Native (Divide & Conquer)
Convergence stability	Handled (sign tracking)	Stable

Both methods yield mathematically equivalent subspaces. While the NumPy implementation exhibits higher execution speed due to compiled backend routines, our pure Python implementation serves the pedagogical objective of exposing all underlying numerical steps.

4 Dataset and Experimental Results

4.1 Data Preprocessing

The model was evaluated on the NSL-KDD network traffic benchmark. Preprocessing was handled via a dedicated pipeline (`dataPrep.py`). Categorical attributes (such as protocol types TCP or UDP) were transformed using One-Hot Encoding. Min-Max normalization was applied across all continuous features; unscaled numerical attributes with large dynamic ranges (such as byte counts) would otherwise disproportionately distort distance metrics in SVD space.

4.2 Evaluation Metrics

Configuring the model to retain $k = 5$ principal components yielded the following results:

- **Overall Accuracy:** 93.80%
- **Detected Attacks:** 233,611 out of 250,436
- **False Positives:** 2,460

5 Result Interpretation: Visual Analysis and Practical Implications

5.1 Graphical Analysis

The visual inspection of the reconstruction error distributions illustrates the separation capability of the SVD subspace.

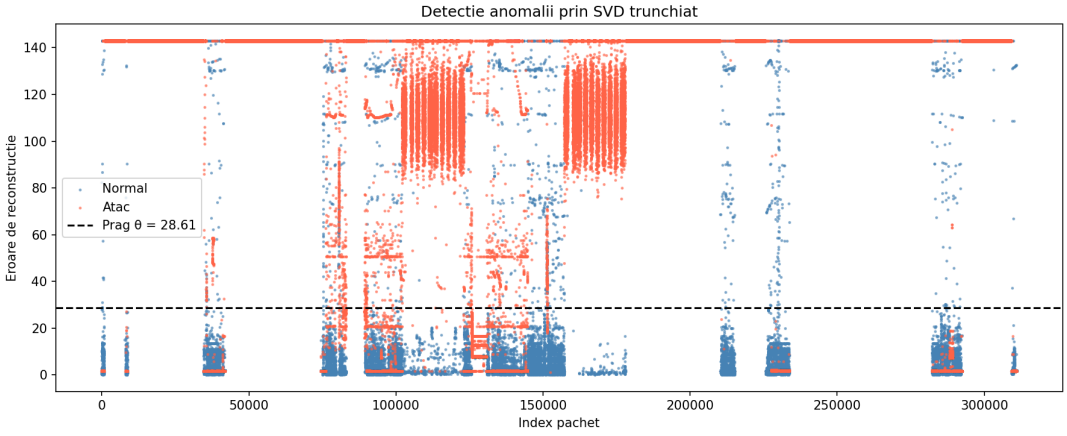


Figure 2: Normal network traffic clusters near zero on the reconstruction error axis, whereas malicious connections induce substantially higher reconstruction errors, allowing clear threshold-based separation.

5.2 Strengths and Limitations

Strengths:

- The model operates on behavioral deviation, enabling the detection of previously unseen zero-day attacks that do not conform to baseline normal patterns.
- Power iteration with truncation avoids the memory overhead of computing full orthogonal matrices, terminating after computing only k components.

Limitations:

- Contamination of the training split with malicious traffic causes anomalous patterns to be integrated into the normal subspace, reducing detection sensitivity.
- Structural shifts in legitimate network usage (concept drift) require model retraining to avoid elevated false-positive rates.

6 Conclusion

This work demonstrates how fundamental linear algebra concepts can be applied to network anomaly detection. Implementing SVD without high-level library abstractions required addressing real-world numerical challenges, including convergence stabilization and overflow prevention via matrix scaling. Achieving an accuracy of nearly 94% on the NSL-KDD benchmark demonstrates the practical viability of subspace reconstruction for anomaly detection.